

Gymnázium J. K. Tyla



Praktický dopad AI na softwarové inženýrství - AI agenti a agentické inženýrství

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Práce zkoumá přechod od konverzační asistence k delegovanému agentnímu vývoji a roli harnessu jako běhového a integračního prostředí coding agenta. Cílem je vysvětlit, jak propojení jazykového modelu s nástroji, stavem, verifikací a lidskými kontrolními body umožňuje řízené provádění softwarových úloh v praxi. Agentické inženýrství práce vymezuje jako soubor postupů, které činí AI-asistovaný vývoj účinným, kontrolovaným, opakovatelným a škálovatelným. Teoretická část vychází z odborných publikací a dokumentace nástrojů. Praktická část analyzuje systém DarkFactory prostřednictvím statické inspekce workflow, runneru, kontejnerového prostředí, deklarativního grafu a testů v přesně určené revizi [e9c10221](#). Analýza dokládá fázované řízení požadavku, schvalovací brány, izolaci změn, automatické spouštění testů, seberevizi a obnovu běhu po vyčerpání kvóty. Výsledky ukazují, že praktická autonomie nevzniká pouze schopnostmi modelu, ale především návrhem harnessu a přesným rozdělením odpovědnosti mezi automatizované mechanismy a lidské rozhodování. Dostupná evidence však neměří kvalitu práce různých modelů ani provozní úspěšnost systému. Závěry jsou proto omezeny na strukturální vlastnosti této jediné reprodukovatelně pinované revize.

Klíčová slova: jazykové modely; coding agents; harness; Agentické inženýrství; GitHub Actions; DarkFactory

Abstract

This thesis examines the transition from conversational assistance to delegated agentic software development and the role of the harness as a coding agent's execution and integration environment. Its objective is to explain how connecting a language model to tools, persistent state, verification, and human control points enables governed execution of software tasks. Agentic Engineering is defined as the set of practices that makes AI-assisted development effective, controlled, repeatable, and scalable. The theoretical part draws on research literature and first-party tool documentation. The practical part analyses DarkFactory through static inspection of the workflow, runner, container environment, declarative graph, and tests at the exact [e9c10221](#) revision. The analysis identifies staged request handling, approval gates, change isolation, automated test execution, self-review, and recovery after quota exhaustion. The findings indicate that practical autonomy does not arise from model capability alone, but primarily from harness design and an explicit division of responsibility between automated mechanisms and human decisions. The evidence does not measure model quality or operational success rates. The conclusions are therefore limited to structural properties of this single reproducibly pinned revision.

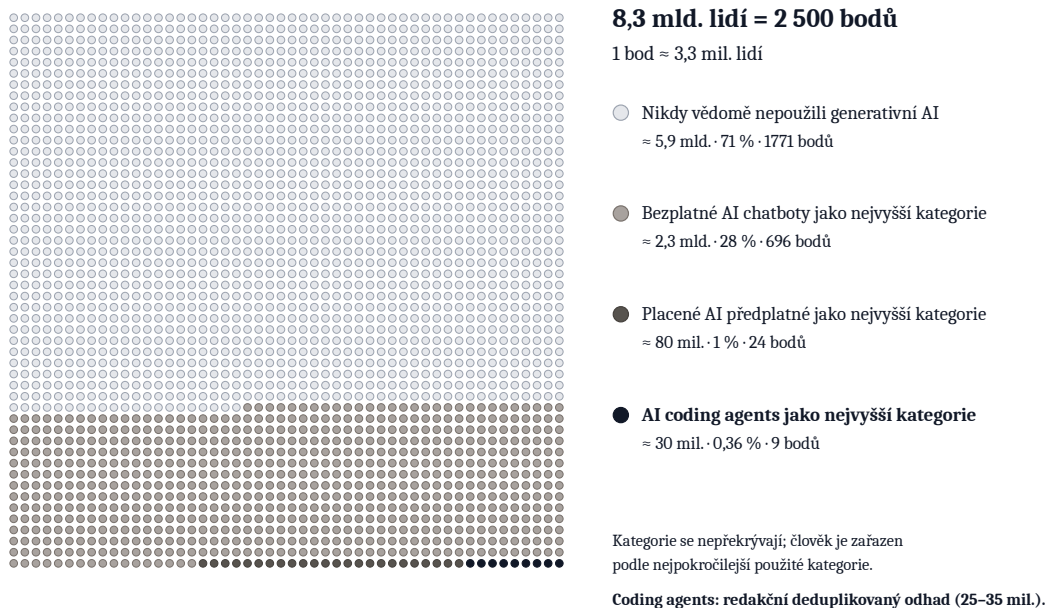
Keywords: language models; coding agents; harness; Agentic Engineering; GitHub Actions; DarkFactory

Obsah

Prohlášení	2
1 Úvod	5
1.1 Cíl a vymezení	5
2 Teoretická část	6
2.1 Jazykový model v agentním systému	6
2.1.1 Context window a kompakce	6
2.2 Agent a harness	7
2.2.1 Smyčka	7
2.3 Agentické inženýrství	8
2.3.1 Zadáání a kontext	8
2.3.2 Orchestrace a lidská integrace	8
3 Praktická část	11
3.1 Metodika	11
3.2 DarkFactory	12
3.2.1 Spuštění a kontejnerizované prostředí	12
3.2.2 Fázovaný průchod a řídicí smyčka	13
3.2.3 Serializace stavu, checkpointing a obnova	13
4 Výsledky a diskuse	14
4.1 Zjištění	14
4.2 Diskuse	15
5 Závěr	16
Seznam zdrojů	17
Seznam obrázků a tabulek	19

1 Úvod

Nástroje založené na velkých jazykových modelech prošly rychlým vývojem: od doplňování kódu při psaní v editoru přes konverzační chatboty až po autonomní agenty, kteří pomocí nástrojů samostatně provádějí změny a spouštějí příkazy v běhovém prostředí (1; 2; 3; 4; 5). S rostoucími schopnostmi modelů roste i jejich adopce, avšak většina uživatelů nemá představu čeho tyto nástroje jsou skutečně schopny. Většina populace se s generativní AI setkává jenom na povrchu prostřednictvím chatbotů a to bezplatným tedy omezeným přístupem. Odhad zdroje klade počet uživatelů chatbotů na 28% populace zatímco pravidelné užití AI coding agents pouze na 0,36 % světové populace(6).



Obrázek 1: Odhad rozdělení uživatelů generativní AI podle typu. Hodnota 30 milionů pravidelných uživatelů coding agents, přibližně 0,36 % světové populace, je redakční střed odhadovaného rozmezí 25–35 milionů (6). Aby mohl agent samostatně pracovat na projektu, nestačí pouhé generování odpovědí. Potřebuje kontext z repozitáře, přístup k prostředí, nástroje pro spouštění příkazů, uchování stavu mezi jednotlivými kroky a vymezený bod, v němž člověk rozhodne o přijetí výsledku (7; 8).

1.1 Cíl a vymezení

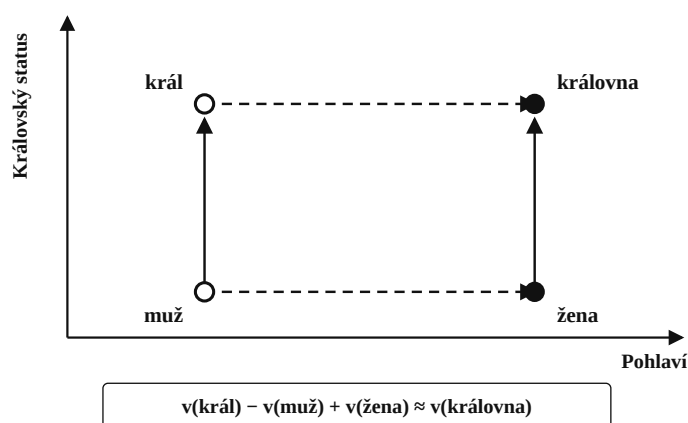
Cílem práce je ukázat, jak harness dělá z jazykového modelu autonomního agenta a jaké postupy umožňují využívat agenty účinně a kontrolovaně. Praktická část analyzuje DarkFactory, na poskytovateli nezávislou pipeline pro AI-asistovaný softwarový vývoj. Pipeline kombinuje workflow GitHub Actions, kontejnerizovaný pythonovský řadič a harness `df` s deklarativním grafovým jádrem v TypeScriptu.

2 Teoretická část

2.1 Jazykový model v agentním systému

Jazykový model (LLM) předpovídá další token na základě předešlých obsaženém v **kontextu**. **Transformer** využívá mechanismu attention (9; 10) pro zpracování vztahů mezi jednotlivými tokeny. Při inferenci model zpracuje obsah kontextového okna a vytvoří posloupnost výstupních tokenů. Jednotlivé volání však samo nemění soubory, nespouští příkazy ani neuchovává stav mezi kroky. Tyto činnosti zajišťuje harness, který modelu zpřístupňuje nástroje a pomocí rekursivního procesu zvaného ReAct dělá z jednotné generace souvislou konverzaci (11).

Vektorové reprezentace, označované jako **embeddingy**, zachycují sémantické vztahy ve vektorovém prostoru. Známým příkladem je vztah mezi vektory slov král, královna, muž a žena (12). Tento vztah schematicky znázorňuje Obrázek 2.



Obrázek 2: Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov **král**, **královna**, **muž** a **žena** (12).

2.1.1 Context window a kompakce

Kontextové okno (**context window**) tvoří pracovní kontext jednoho volání modelu. Může obsahovat instrukce, části repozitáře, historii volání nástrojů i výsledky předchozích kroků. Jeho kapacita však sama o sobě nezaručuje, že model všechny podstatné informace správně využije: úspěšnost jejich vybavení závisí také na umístění v kontextu a může s rostoucí délkou vstupu klesat (13). Toto postupné zhoršování práce s nahromaděným kontextem se označuje jako **context rot** (14).

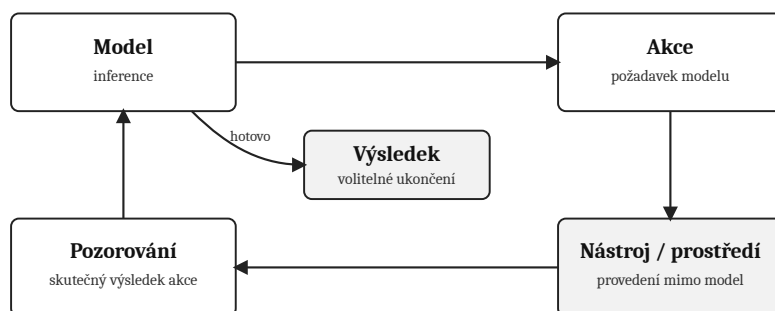
Kompakce (**compaction**) po překročení stanoveného limitu nahrazuje starší průběh strukturovaným souhrnem klíčových rozhodnutí a dosažených výsledků. Do dalšího volání tak není nutné vkládat celý přepis předchozí interakce (14).

2.2 Agent a harness

Praktickou hranicí mezi konverzačním chatbotem a agentem je míra delegovaného provádění. V běžném chatovém režimu model především vrací text a uživatel zůstává vykonavatelem navržených kroků. Agent naproti tomu prostřednictvím harnessu získává řízený přístup k nástrojům a prostředí: může procházet soubory, upravovat kód, spouštět testy a používat jejich výstupy v dalších krocích (11; 15). Role člověka se tím může posunout od provádění jednotlivých kroků k zadání, omezení a revizi delegované práce.

2.2.1 Smyčka

Základním mechanismem agentického systému je iterativní řídicí smyčka podle vzoru **ReAct (Reasoning and Acting)** (16). Model v každém kroku na základě aktuálního kontextu zvolí mezi možnostmi rozvažování, volání nástroje ve strukturovaném požadavku nebo zprávy v chatu (konečná akce). Harness každý krok přidá do přepisu konverzace a nový přepis se vrátí modelu. V případě použití nástroje tuto akci provede v běhovém prostředí, zachytí výsledek a vrátí jej modelu jako nové pozorování.



Obrázek 3: Agentní smyčka ReAct: model navrhne akci, harness ji zprostředkuje a vykoná v prostředí a pozorování se vrací do dalšího kroku(16).

2.3 Agentické inženýrství

Agentické inženýrství (**Agentic Engineering**) se snaží definovat přístup aplikovatelný pro jakékoliv AI-asistované inženýrské práce, označuje jak systematický design systému a přístupu k práci tak samotné AI-asistované inženýrství v jakémkoliv kontextu. Mezi hlavní oblasti patří: **Prompt Engineering**, **Context Engineering**, **Harness Engineering**, **Loop Engineering** a **Workflow/Graph Engineering** (7; 14; 15; 17; 18). Cílem je, aby vývojář mohl efektivně a kontrolovaně delegovat dílčí úkoly agentovi, aniž by ztratil přehled o záměru, rozsahu a kvalitě výsledku.

2.3.1 Zadání a kontext

Spolehlivé delegování práce začíná explicitním vymezením cíle, rozsahu, omezení a podmínek přijetí. Specifikace popisuje nejen požadovaný výsledek, ale také části systému, které se měnit nemají, a způsob, jakým bude výsledek ověřen. Tento přístup, označovaný jako **spec-first** nebo **spec-driven development**, dává agentovi před implementací měřitelné hranice a člověku podklad pro posouzení výsledku, je také používán v klasickém vývoji softwaru.

Prompt engineering se soustředí na formulaci instrukcí, omezení, příkladů a očekávaného výstupu konkrétního inferenčního kroku (17). **Context engineering** řeší širší a průběžný výběr, uspořádání, obnovování a kompakci informací, které má model v daném kroku k dispozici (14).

Nástroje umožňují agentovi číst a upravovat soubory, vyhledávat nebo spouštět příkazy; **Skills** spojují opakovaně použitelné instrukce, skripty a zdroje pro určitý typ úlohy (19). **Hooks** reagují na události životního cyklu a mohou před akcí či po ní vynutit deterministickou kontrolu (20). **Model Context Protocol (MCP)** standardizuje napojení externích nástrojů a datových zdrojů prostřednictvím rozhraní klient-server (21). Instrukce lze uchovat ve standardizovaném souboru `AGENTS.md` (22) přímo v repozitáři (Anthropic ojedinele využívá **CLAUDE.md**). Skilly skripty a hooky lze uchovat pod složkou `.agents/` v projektu nebo v konfigurační složce harnessu.

2.3.2 Orchestrace a lidská integrace

Pokud mají podúlohy jasné hranice a jejich výsledky lze znovu integrovat, rozsáhlou úlohu lze rozdělit do více agentních běhů. Ve vzoru **coordinator/subagent** koordinátor deleguje dílčí úkol specializovanému subagentovi s vlastním kontextem a přebírá jeho výsledek. Nezávislé podúlohy mohou zpracovat paralelní pracovníci. **Workflow graph** předem určuje závislosti, pořadí a větvení fází (18). **Agent Swarm** dynamicky rozkládá úlohu na heterogenní podproblémy a spouští specializované agenty paralelně pod řízením orchestrátoru (23).

Více agentů samo o sobě nezaručuje lepší výsledek. Paralelizace přináší užitek jen tehdy, když jsou omezeny vzájemné závislosti a koordinátor dokáže odhalit konflikty, ověřit dílčí výstupy a posoudit sloučený výsledek vůči společným podmínkám přijetí. Orchestrace proto zahrnuje nejen rozdělení práce, ale také správu kontextu, pořadí kroků, sdíleného stavu a integračních kontrol. Orchestrace je však nezbytná pro rozsáhlé úlohy, které by nebylo možné implementovat v rozsahu jednoho kontextového okna, spolehání na kompakci by vedlo ke katastrofické divergenci a kdybyse soustředil na celek nebyl by schopen efektivně implementovat jednotné části zadání.

Goal loop označuje nadřazenou řídicí smyčku: po dílčím dokončení ReAct smyčky harness porovná pozorovaný stav s cílem a podmínkami přijetí a podle výsledku běh ukončí, nebo zahájí další iteraci či změnu strategie. (16).

human-in-the-loop (HITL) označuje bod kde se do automatizované smyčky doplní kontrolní brány, v nichž je vyžadováno explicitní lidské rozhodnutí, například schválení specifikace, potvrzení implementačního plánu nebo přijetí výsledného diffu. Vývojář tak může ponechat agentovi ohraničenou implementační práci a současně si zachovat odpovědnost za záměr a integraci.

PRAKTICKÁ ČÁST

DarkFactory:

**Pipeline pro automatizované
AI-asistované softwarové inženýrství**

3 Praktická část

3.1 Metodika

Praktická část navazuje na teoretická východiska o harnessu a Agentickém inženýrství. Zkoumá, jak jsou principy řízeného provádění, ohraničení kontextu a verifikace realizovány ve zdrojovém kódu DarkFactory.

Analýza se zaměřuje na čtyři klíčové inženýrské dimenze odvozené z teoretické části:

1. **Izolace běhového prostředí a oprávnění:** kontejnerizované oddělení agentního procesu od hostitelského shellu CI, explicitní připojení pracovního stromu a předávání autentizačních údajů pouze do kroků, které je používají.
2. **Dekompozice úlohy a fázované řízení (HITL):** rozdělení životního cyklu požadavku do diskrétních stavů (interpretace, plánování, implementace) a začlenění formálních schvalovacích bran člověkem.
3. **Automatizovaná verifikace a seberevize:** spuštění dostupných formátovacích nástrojů a deklarovaných testovacích sad, předání chybového výstupu jednomu oprávněnému kroku a kontrola změněných souborů a diffu vůči schválenému plánu.
4. **Perzistence stavu a zotavení:** serializace parametrů běhu (checkpointing), zachyt výpadků kvót externích API (HTTP 429) a procedura obnovení rozpracované úlohy příkazem /resume.

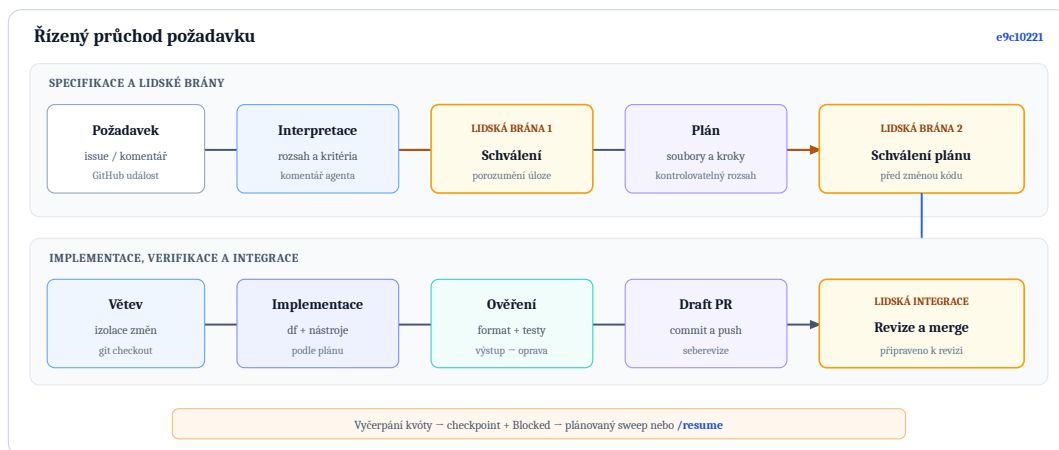
Metodou je trasovatelná statická inspekce workflow `.github/workflows/agent.yml`, obrazu `docker/Dockerfile.agent`, produkčního řadiče `.github/scripts/agent_runner.py`, grafu `harness/assets/graph.darkfactory.json`, jeho ukládání stavu v `harness/src/graph/run-state.ts` a souvisejících testů. Zjištění popisují mechanismy, které jsou v revizi přítomné a testované; nepředstavují měření provozní úspěšnosti ani srovnání kvality modelů.

Revize obsahuje dvě související, ale nezaměnitelné vrstvy orchestrace. Produkční workflow vstupuje do pythonovského řadiče, který volá příkaz `df run`. Vedle něj je v repozitáři deklarativní grafové jádro v TypeScriptu s uzly, hranami, bezpečnostními rozpočty smyček a atomicky ukládaným stavem `df`. Práce proto neprezentuje grafové jádro jako vstupní bod produkčního workflow, ale hodnotí jej jako samostatnou architektonickou součást stejného snapshotu (24).

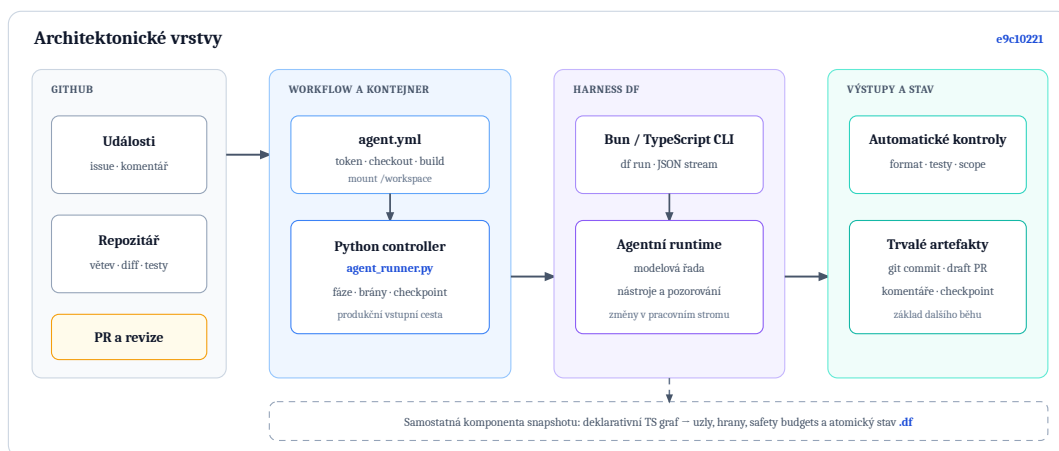
3.2 DarkFactory

DarkFactory představuje automatizovanou pipeline běžící v prostředí GitHub Actions. Workflow spouští kontejner Docker, v němž pythonovský runner připraví prostředí, předá harnessu instrukce a zpracuje výsledek jeho běhu (24).

Produkční průchod požadavku shrnuje Obrázek 4 a vztah workflow, řadiče, harnessu a odděleného grafového jádra znázorňuje Obrázek 5.



Obrázek 4: Produkční průchod požadavku v hodnocené revizi DarkFactory [e9c10221]: lidské brány oddělují interpretaci, plán a integraci výsledku (24).



Obrázek 5: Architektonické vrstvy hodnocené revize: produkční GitHub Actions/Python cesta používá harness df; deklarativní grafové jádro v TypeScriptu je v témže snapshotu přítomno jako samostatná komponenta (24).

3.2.1 Spuštění a kontejnerizované prostředí

Produkční cesta reaguje na události GitHubu, zejména otevření issue a přidání komentáře; workflow navíc podporuje review komentáře, repository_dispatch a explicitní či znovu-použitelné spuštění. Job je podmíněn povolením agenta pro repozitář a před nákladnějšími kroky ignoruje komentáře automatických botů a položky označené jako selhání pipeline. Po validaci vstupu workflow sestaví obraz Docker podle docker/Dockerfile.agent. Pracovní kopie cílového repozitáře je do kontejneru připojena jako svazek /workspace; runner

tedy pracuje nad explicitně předaným pracovním stromem místo přímého běhu v hostitelském shellu GitHub Actions. Potřebné tokeny a modelové přihlašovací údaje jsou kontejneru předány proměnnými prostředí. Samotné workflow deklaruje zápisová oprávnění pro obsah repozitáře, issues, pull requesty, projekty a Actions, protože tyto operace pipeline skutečně provádí (24).

3.2.2 Fázovaný průchod a řídicí smyčka

Řízení úlohy v kontejneru přebírá skript `agent_runner.py`. Průchod není realizován jako jediná dlouhá interakce, nýbrž je rozdělen do fází oddělených kontrolními branami:

- **Fáze interpretace:** Runner načte text issue a předloží jej modelu se systémovou instrukcí pro dekompozici problému. Model zformuluje pochopení úlohy a navrhne akceptační kritéria. Runner výsledek zapíše jako komentář k issue a ukončí běh, čímž vyčká na schválení člověkem.
- **Fáze plánování:** Po explicitním schválení interpretace je workflow spuštěno znovu. Model analyzuje strukturu repozitáře a sestaví konkrétní plán úprav — specifikuje seznam souborů určených ke změně a předpokládaný sled kroků. Tento plán je opět předložen člověku ke schválení.
- **Implementační smyčka a verifikace:** Teprve po schválení plánu runner vytvoří dedikovanou pracovní větev, aby nezasahoval do hlavní vývojové větve. Harness následně model navádí k úpravě kódu. Runner poté spustí dostupné formátovací nástroje a deklarované testovací sady: Python testy při přítomnosti `pyproject.toml`, Cargo testy pro Rust workspace a kořenový `test` skript pro Node/Bun projekt. Při prvním neúspěchu předá chybový výstup jednomu oprávněmu kroku modelu. V hodnocené revizi však po tomto oprávněm kroku před commitem povinně nespouští celou testovací sadu znovu; jde proto o omezení produkční cesty, nikoli o end-to-end důkaz zeleného opraveného stromu.
- **Seberevize a draft pull request:** Po implementační fázi runner vytvoří commit, odešle větev a otevře draft pull request. Seberevize nejprve deterministicky porovná změněné soubory s explicitním file-scope plánem, pokud jej plán obsahuje, a následně předá diff modelu ke code review. Čistá seberevize pokračuje samostatnou modelovou kontrolou souladu výsledného diffu s plánem. Teprve kladný výsledek této kontroly označí pull request jako připravený k lidské revizi (24).

3.2.3 Serializace stavu, checkpointing a obnova

V distribuovaném prostředí automatizovaných pipeline mohou běh přerušit limity externích služeb nebo jiné provozní chyby. Hodnocená produkční cesta explicitně řeší jeden konkrétní případ: úplné vyčerpání dostupné modelové řady kvůli kvótě, při němž vytvoří checkpoint pro pozdější obnovení.

Pythonovský runner nezapisuje checkpoint průběžně po každém kroku. Pokud se vyčerpají všechny dostupné modelové pokusy, uloží `.antigravity_checkpoint.json` s identifikátorem položky, větví a dosud dokončenými kroky, případné rozpracované změny commitne na pracovní větev, položku označí jako blokovanou a zapíše čas obnovení. Samostatný workflow `quota-resume.yml` provádí každých 15 minut sweep zralých záznamů; alternativou je příkaz `/resume`. Deklarativní grafové jádro ukládá svůj `RunState` odděleně do souboru `.df` atomickou náhradou. Jde tedy o dva rozdílné mechanismy perzistence v témže snapshotu (24).

4 Výsledky a diskuse

4.1 Zjištění

Pipeline organizuje průchod požadavku jako sled událostí v repozitáři. Po vytvoření issue runner vygeneruje interpretaci zadání; její schválení uživatelem iniciuje tvorbu plánu a teprve další schválení zahájí samotnou implementaci. Dvě schvalovací brány tak oddělují porozumění požadavku a plánování od samotných úprav zdrojového kódu (24).

Workflow před spuštěním ověřuje, zda je agent pro daný repozitář povolen, a ignoruje komentáře vytvořené automatickými boty. Autentizační tokeny jsou předány z prostředí GitHub Actions a pracovní strom projektu je připojen do kontejneru, v němž běží pythonovský runner (24).

Po schválení plánu pipeline vytvoří pracovní větev a předá implementační instrukci harnessu. Runner spustí dostupné formátovací nástroje a deklarované testovací sady; při prvním neúspěchu předá jejich výstup jednomu opravnému kroku. V této revizi však po opravě povinně neopakuje celou testovací sadu před commitem. Následně otevře draft pull request, provede deterministickou kontrolu explicitního file-scope a modelovou seberevizi a při čistém výsledku ještě samostatně porovná finální diff s plánem. Teprve poté označí pull request jako připravený k lidské revizi (24).

Inspekce testovací sady ukazuje pokrytí ukládání a atomické obnovy grafového stavu, plánování a autorizace bran, scope kontroly, plánovaného quota-resume sweepu a failoveru mezi poskytovateli. Tyto testy ověřují konkrétní řídicí mechanismy v izolaci; samy o sobě neprokazují provozní úspěšnost celé pipeline ani to, že model po opravném kroku odstraní lokální regresi (24).

Při úplném vyčerpání modelové řady runner uloží quota checkpoint, označí položku jako blokovanou a zapíše repository variable s časem obnovení. Běh lze znovu vyvolat příkazem `/resume` nebo plánovaným sweepem; samostatné testy ověřují zralý i nezralý quota záznam, neúspěšný dispatch a zachování záznamu pro další pokus (24).

4.2 Diskuse

Pinovaný snapshot dokládá mechanismy trvalého stavu, oprávnění, izolace změn, automatických kontrol a obnovy. Neměří však účinnost jednotlivých postupů Agentického inženýrství, kvalitu práce s kontextem ani provozní úspěšnost orchestrace. Následující interpretace jsou proto omezeny na strukturální vlastnosti revize [e9c10221](#).

Dělbá odpovědnosti mezi modelem a harnessem. Samostatné volání jazykového modelu nemá bez dodaného kontextu a nástrojů trvalý přístup ke stavu vývojového prostředí. Hodnocený systém proto rozděluje odpovědnost mezi model a řídicí vrstvu: GitHub Actions a pythonovský runner připravují prostředí, určují pořadí fází, předávají nástrojové výstupy a vykonávají automatické kontroly, zatímco model zajišťuje interpretaci, implementační návrhy, opravy a části revize. Harness zde není čistě deterministický; kombinuje deterministické mechanismy s modelovými kroky v explicitně řízeném procesu (7; 24).

Architektonický kompromis: autonomie vs. správa. Produkční cesta kombinuje dvě lidská schválení před implementací s automatickým spuštěním testů, scope kontrolou, modelovou seberevizí, kontrolou souladu s plánem a následnou lidskou revizí pull requestu. Deklarativní graf v témže snapshotu navíc modeluje samostatné deviation a merge gates. Tato vícevrstvá kontrola omezuje riziko driftu zadání a změn mimo schválený rozsah, nezaručuje však jejich odstranění. Cenou je více přerušení autonomie a závislost na lidských rozhodnutích (8; 24).

Kontext a perzistence stavu. V souladu s poznatky o degradaci vybavení informací v dlouhém kontextu (13) DarkFactory externalizuje významnou část pracovního stavu do git/GitHub artefaktů a při vyčerpání kvóty také do checkpointu. Jednotlivé agentní fáze dostávají účelově sestavené prompty s požadavkem, plánem, diffem nebo bezprostředním chybovým výstupem podle daného kroku. Statická inspekce však neměří skutečnou délku kontextu, tokenové náklady ani kvalitu výběru informací; z této architektury proto nelze samostatně odvodit úsporu tokenů ani vyšší přesnost modelu (24).

Deterministické kontroly jako protiváha stochastické generace. Softwarové inženýrství nabízí agentům strojově ověřitelnou zpětnou vazbu testů a dalších projektových kontrol. V hodnocené produkční cestě je deterministické samotné spuštění deklarovaných testovacích sad a file-scope kontrola, zatímco code review a plan alignment znovu používají model. Runner předává první chybový výstup opravnému kroku, ale před commitem po této opravě povinně neopakuje testovací sadu. Snapshot proto dokládá kombinaci deterministických a modelových kontrol, nikoli plně deterministickou end-to-end verifikaci (24).

5 Závěr

Jazykový model sám o sobě pouze generuje výstup v závislosti na vstupním kontextu. Agent z něj vzniká až propojením s nástroji, prostředím, stavem a pozorováním, které zajišťuje harness (7; 11). Ani toto propojení však pro účinné nasazení ve vývoji softwaru nestačí bez postupů Agentického inženýrství, které vymezují rozsah autonomie, určují body lidského rozhodnutí a zajišťují ověřování výsledků.

Analýza pipeline DarkFactory doložila, jak lze takový systém realizovat v praxi: řídí průchod požadavku, vytváří izolovanou větev, spouští projektové testy, kontroluje rozsah změn, při vyčerpání kvóty ukládá checkpoint a předává výsledek k lidské revizi. Zároveň odhalila konkrétní omezení produkčního runneru: po automatickém opravném kroku není v této revizi před commitem vynucen nový úplný běh testovací sady.

Úloha vývojáře se v tomto uspořádání posouvá od rutinního psaní kódu k preciznímu návrhu zadání, vymezení mantinelů a určení bodů, v nichž je nezbytné lidské rozhodnutí. Zjištění ukazují, že samotná autonomie agenta nenahrazuje řízení vývojového procesu. Správně navržený harness spolu s postupy Agentického inženýrství umožňuje delegovat netriviální vývojové úlohy na stochastické modely a současně chránit stabilitu hlavní vývojové větve i transparentnost provedených změn. Rozsah této práce však neumožňuje zobecnit výsledky na jiné harnessy ani posoudit kvalitu kódu vytvářeného různými modely.

Seznam zdrojů

1. GITHUB. *GitHub Copilot code suggestions in your IDE* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/concepts/completions/code-suggestions>
2. GITHUB. *Asking GitHub Copilot questions in your IDE* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/how-tos/chat-with-copilot/chat-in-ide>
3. GITHUB. *Use GitHub Copilot agents* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/how-tos/use-copilot-agents>
4. OPENAI. *Introducing Codex* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-codex/>
5. OPENAI. *Introducing the Codex app* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-the-codex-app/>
6. HILLEBRANDT, Finn. *How Many People Use AI? Facts & Figures 2026* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://www.gradually.ai/en/how-many-people-use-ai/>
7. ANTHROPIC. *Harness design for long-running application development* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
8. ANTHROPIC. *Scaling Managed Agents: Decoupling the brain from the hands* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://www.anthropic.com/engineering/managed-agents>
9. VASWANI, Ashish, Noam SHAZEER, Niki PARMAR, Jakob USZKOREIT, Llion JONES, Aidan N. GOMEZ, Łukasz KAISER a Illia POLOSUKHIN. Attention Is All You Need. *arXiv preprint arXiv:1706.03762* [online]. 2017. Dostupné z: <https://arxiv.org/abs/1706.03762>
10. BROWN, Tom B., Benjamin MANN, Nick RYDER, Melanie SUBBIAH, Jared KAPLAN, Prafulla DHARIWAL, Arvind NEELAKANTAN, Pranav SHYAM, Girish SASTRY, Amanda ASKELL a OTHERS. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems* [online]. 2020, **33**. Dostupné z: <https://arxiv.org/abs/2005.14165>
11. ANTHROPIC. *Building effective agents* [online]. 2024 [vid. 2026-09-23]. Dostupné z: <https://www.anthropic.com/engineering/building-effective-agents>
12. MIKOLOV, Tomas, Wen-tau YIH a Geoffrey ZWEIG. Linguistic Regularities in Continuous Space Word Representations. In: *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* [online]. 2013, s. 746–751. Dostupné z: <https://aclanthology.org/N13-1090/>

13. LIU, Nelson F., Kevin LIN, John HEWITT, Ashwin PARANJAPE, Michele BEVILACQUA, Fabio PETRONI a Percy LIANG. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* [online]. 2024, **12**, 157–173. Dostupné z: <https://arxiv.org/abs/2307.03172>
14. ANTHROPIC. *Effective context engineering for AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
15. OPENAI. *Sandbox agents: Concepts | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-js/guides/sandbox-agents/concepts/>
16. YAO, Shunyu, Jeffrey ZHAO, Dian YU, Nan DU, Izhak SHAFRAN, Karthik NARASIMHAN a Yuan CAO. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629* [online]. 2022. Dostupné z: <https://arxiv.org/abs/2210.03629>
17. OPENAI. *Prompt engineering* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://platform.openai.com/docs/guides/prompt-engineering>
18. OPENAI. *Agent orchestration | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-21]. Dostupné z: https://openai.github.io/openai-agents-python/multi_agent/
19. AGENT SKILLS. *Agent Skills: an open format for extending AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://agentskills.io/home>
20. OPENAI. *Lifecycle | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-python/ref/lifecycle/>
21. MODEL CONTEXT PROTOCOL. *Model Context Protocol 2026-07-28 specification: server features* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://modelcontextprotocol.io/specification/2026-07-28/server>
22. AGENTIC AI FOUNDATION. *AGENTS.md* [online]. 2026 [vid. 2026-09-23]. Dostupné z: <https://agents.md/>
23. KIMI TEAM. Kimi K2.5: Visual Agentic Intelligence. *arXiv preprint arXiv:2602.02276* [online]. 2026 [vid. 2026-09-23]. Dostupné z: <https://arxiv.org/abs/2602.02276>
24. MARIUS, Patrik. *DarkFactory commit e9c10221* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://github.com/marius-patrik/DarkFactory/commit/e9c10221b40589512d262a0edb95f709b923150c>

Seznam obrázků a tabulek

Obrázek 1	Odhad rozdělení uživatelů generativní AI podle typu. Hodnota 30 milionů pravdělných uživatelů coding agents, přibližně 0,36 % světové populace, je redakční střed odhadovaného rozmezí 25–35 milionů (6).....	5
Obrázek 2	Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov král , královna , muž a žena (12).....	6
Obrázek 3	Agentní smyčka ReAct: model navrhne akci, harness ji zprostředkuje a vykoná v prostředí a pozorování se vrací do dalšího kroku(16).....	7
Obrázek 4	Produkční průchod požadavku v hodnocené revizi DarkFactory <code>e9c10221</code> : lidské brány oddělují interpretaci, plán a integraci výsledku (24).....	12
Obrázek 5	Architektonické vrstvy hodnocené revize: produkční GitHub Actions/Python cesta používá harness <code>df</code> ; deklarativní grafové jádro v TypeScriptu je v témže snapshotu přítomno jako samostatná komponenta (24).....	12